

Assignment 3 - Style Transfer

Danial Bahrami Siavashani

CSCI 611 – Applied Machine Learning

Instructor: Dr. Bo Shen

Submission Date: April 26, 2026

1. Introduction

This assignment focused on implementing neural style transfer using a pretrained convolutional neural network. The goal was to take two images — a content image and a style image — and generate a new image that preserves the structure of the content while applying the visual appearance of the style. This was done by following the method described in the paper by Gatys et al. (2016).

Unlike Assignment 2 where the focus was on training a CNN for classification, this assignment used a pretrained VGG19 network purely as a feature extractor. Instead of updating the model weights, the optimization was applied directly to the pixel values of the target image. This was an interesting shift in perspective and helped me understand how CNN features can be used beyond classification tasks.

In this report, I go over the implementation details, the experiments I ran with different images and hyperparameter settings, and what I observed from the results.

2. Background

Neural style transfer is a technique that combines the content of one image with the visual style of another. It works by using the internal representations of a pretrained CNN to separately capture what an image contains and how it looks. These two things are then combined into a new image through an optimization process.

This assignment was different from previous ones because no training happened in the traditional sense. The VGG19 network was already trained on ImageNet and its weights were frozen throughout. The only thing being optimized was the target image itself, which made this a unique application of deep learning concepts covered in the course.

2.1 Neural Style Transfer Overview

Neural style transfer works by passing images through a deep CNN and extracting feature representations at different layers. Early layers capture low-level details like edges and textures, while deeper layers capture higher-level content like objects and structure. Style transfer takes advantage of this by using shallow layers to capture the style of a painting and a deeper layer to capture the content of a photograph. The result is a new image that looks like the photograph was painted in the style of the artwork.

2.2 Gatys et al. Paper Summary

The method used in this assignment is based on the paper "Image Style Transfer Using Convolutional Neural Networks" by Gatys, Ecker, and Bethge (2016). The paper introduced the idea of using a pretrained VGG network to separate image content from style. Content is represented by the feature activations at a specific layer (conv4_2), while style is represented using Gram matrices computed across multiple layers. A new image is generated by starting from the content image and iteratively updating its pixels to minimize a combined content and style loss. The paper showed that this approach could produce visually impressive results that matched the appearance of well-known artworks.

2.3 Key Concepts: Content Loss, Style Loss, Gram Matrix, Total Loss

Content Loss measures how different the target image is from the content image at layer conv4_2. It is calculated as the mean squared difference between their feature maps at that layer. Minimizing this keeps the target image recognizable as the original photo.

Gram Matrix is used to represent style. It captures the correlations between different feature maps at a given layer by multiplying the feature map matrix by its transpose. This destroys spatial information but keeps texture patterns, which is exactly what defines visual style.

Style Loss is calculated by comparing the Gram matrices of the target image and the style image across multiple layers. The difference between them at each layer is weighted and summed to form the total style loss.

Total Loss is the weighted combination of content and style loss: $L_{\text{total}} = \alpha \times L_{\text{content}} + \beta \times L_{\text{style}}$. The ratio between α and β controls how much the final image looks like the content photo versus the style painting.

3. Implementation Details

3.1 Completed TODOs / Main Code Changes

The notebook provided four main TODOs that needed to be completed. Below is a description of each along with the implemented code.

TODO 1 — Content and Style Images

The first TODO required selecting a content image and a style image. The content image represents the photograph whose structure we want to preserve, and the style image represents the artwork whose texture and appearance we want to transfer. Both images are loaded using the `load_image` helper function which resizes them and normalizes them to match VGG19's expected input format.

```
content = load_image('my.content.image.jpg').to(device)
style = load_image('m.style.image.jpg', shape=content.shape[-2:]).to(device)
```

TODO 2 — Layer Mapping in `get_features()`

The second TODO required mapping PyTorch's VGG19 layer indices to the layer names used in the Gatys et al. paper. The paper specifies using `conv4_2` for content representation and five layers (`conv1_1` through `conv5_1`) for style representation. The indices were determined by printing the VGG19 architecture and counting the Conv2d layers.

```
if layers is None:
    layers = {'0': 'conv1_1', <- style
             '5': 'conv2_1', <- style
             '10': 'conv3_1', <- style
             '19': 'conv4_1', <- style
             '21': 'conv4_2', <- content
             '28': 'conv5_1' <- style
            }
```

TODO 3 — Gram Matrix

The third TODO required implementing the Gram matrix function. The Gram matrix captures style by computing correlations between feature maps at a given layer. Spatial information is destroyed in this process, which means the result captures texture patterns without encoding where in the image those patterns appear.

```
def gram_matrix(tensor):
    _, d, h, w = tensor.size()
    tensor = tensor.view(d, h * w)
    gram = torch.mm(tensor, tensor.t())
    return gram
```

TODO 4 — Content, Style, and Total Losses

The final TODO required implementing the three loss functions inside the training loop. The content loss is the mean squared difference between the target and content feature maps at `conv4_2`. The style loss accumulates the weighted mean squared difference between Gram matrices at each style layer, normalized by layer size. The total loss combines both using the alpha and beta weights from the paper.

```
# Content loss
target_features = get_features(target, vgg)
content_loss = torch.mean((target_features['conv4_2'] -
content_features['conv4_2'])**2)
# Style loss
style_loss = 0
for layer in style_weights:
    # get the "target" style representation for the layer
    target_feature = target_features[layer]
    _, d, h, w = target_feature.shape
    target_gram = gram_matrix(target_feature)
    style_gram = style_grams[layer]
    layer_style_loss = style_weights[layer] * torch.mean((target_gram -
style_gram)**2)
    style_loss += layer_style_loss / (d * h * w)
# Total loss
total_loss = content_weight * content_loss + style_weight * style_loss
```

3.2 Layers Used for Content and Style Representation

VGG19 in PyTorch numbers every operation sequentially from 0 to 36, including Conv2d layers, ReLU activations, and MaxPool layers. The "19" in VGG19 refers only to the 16 convolutional layers plus 3 fully connected layers that have learnable weights — not the total operation count.

The following layers were selected based on the Gatys et al. paper:

PyTorch Index	Layer Name	Role	Why This Layer
0	conv1_1	Style	Low-level textures, fine edges
5	conv2_1	Style	Slightly larger texture patterns
10	conv3_1	Style	Mid-level textures
19	conv4_1	Style	Higher-level patterns
21	conv4_2	Content	High-level object structure
28	conv5_1	Style	Large-scale compositional style

The content layer conv4_2 was chosen because it is deep enough in the network to capture high-level semantic information about what objects are in the image, without being so deep that all spatial detail is lost. The five style layers were chosen to capture texture at multiple scales simultaneously, which is what creates the multi-scale style representation described in the paper.

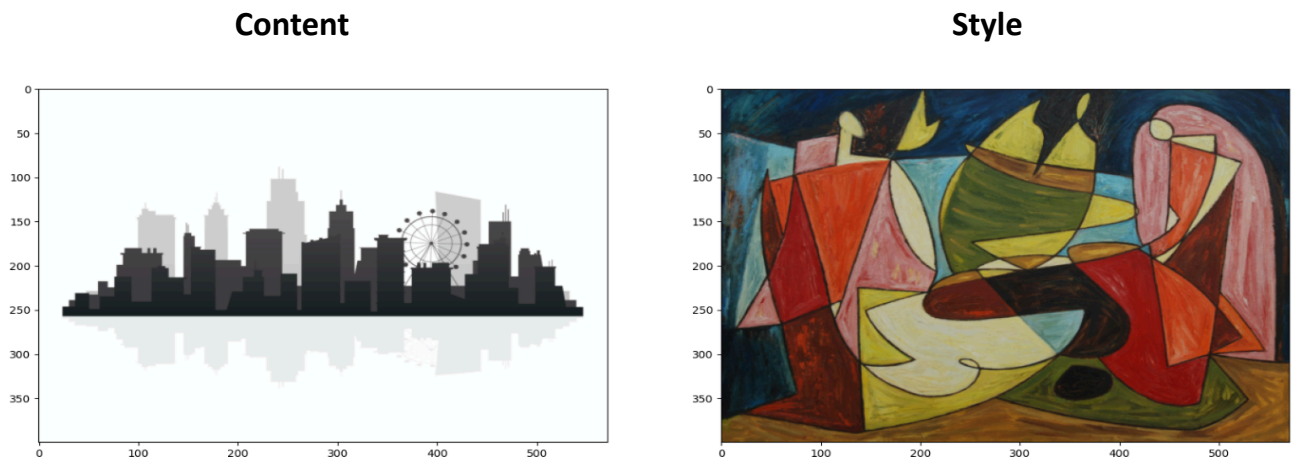
4. Experimental Setup

4.1 Input Content and Style Images

Five runs were conducted across three different content and style image combinations. All images were sourced from free image platforms and resized during loading using the `load_image` function.

Run 1 — City Skyline + Picasso The content image is a city skyline silhouette featuring recognizable urban landmarks including a ferris wheel. The style image is a Picasso painting with bold geometric shapes, strong outlines, and a warm color palette of reds, yellows, and greens. This combination was used as the baseline run with `max_size=400`.

Figure 1. Run 1 content + style side by side



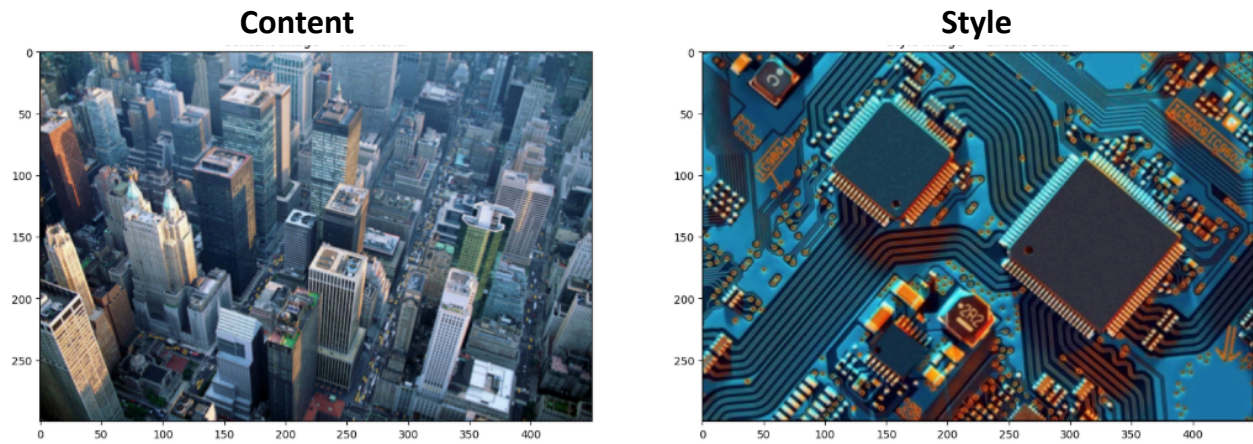
Run 2 — Toronto Flatiron + Starry Night The content image is a photograph of the Flatiron Building in Toronto, Canada, taken in winter with snow on the ground. The style image is *The Starry Night* by Vincent van Gogh (1889), known for its swirling blue brushstrokes and expressive night sky. This combination was chosen to test how the algorithm handles a realistic architectural photograph with a highly textured painting style.

Figure 2. Run 2 content + style side by side



Run 3, 4, 5 — NYC Aerial + Circuit Board The content image is an aerial photograph of Manhattan, New York City, showing a dense grid of skyscrapers from above. The style image is a macro photograph of a circuit board featuring teal and orange geometric patterns, copper traces, and electronic components. This combination was chosen because both images share a similar grid-like structural pattern, making it an interesting non-traditional style transfer experiment.

Figure 3. Run 3,4,5 content + style side by side



4.2 Hyperparameters Tested

The following hyperparameters were used and varied across the five runs:

Hyperparameter	Description	Values Tested
style_weight (β)	Controls how strongly the style is applied	1e6, 1e4
content_weight (α)	Controls how much content structure is preserved	1 (fixed)
steps	Number of optimization iterations	2000, 500
learning_rate	Adam optimizer step size	0.003 (fixed)
max_size	Maximum image dimension in pixels	400, 300
style_weights per layer	Individual weight per style layer	Fixed across all runs

The main variables changed between runs were style_weight and steps. content_weight and learning_rate were kept fixed across all runs as changing too many things at once makes it harder to isolate the effect of each hyperparameter. The per-layer style weights were also kept at their default values (conv1_1=1.0, conv2_1=0.8, conv3_1=0.5, conv4_1=0.3, conv5_1=0.1) throughout all experiments.

5. Results and Findings

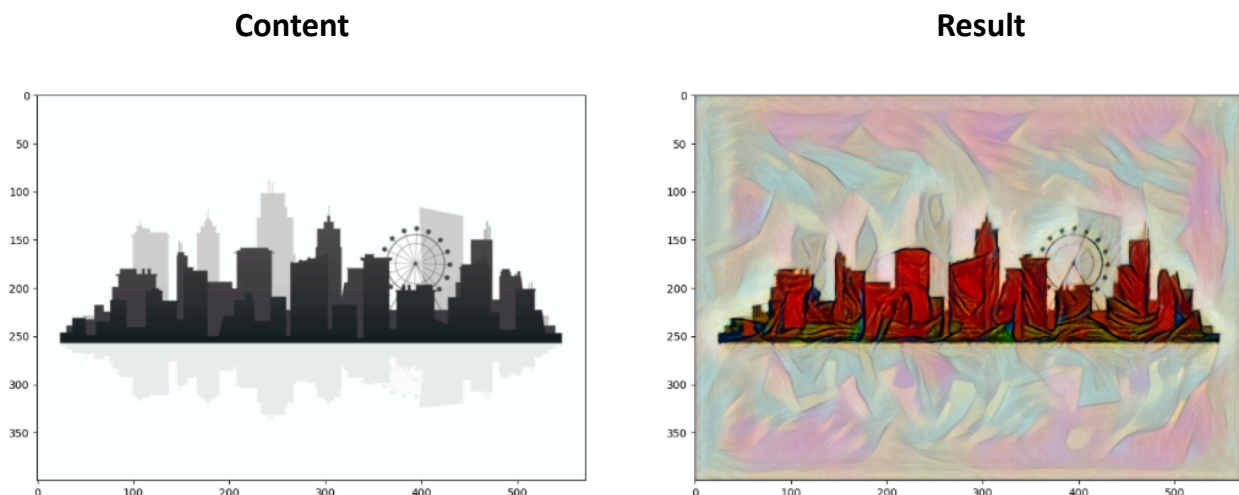
5.1 Baseline Style Transfer Result

Run 1 was the baseline run. I used the city skyline as the content image and a Picasso painting as the style image, with `style_weight=1e6`, `steps=2000`, and `max_size=400`. The final total loss was 13,210,935 and it took 75.63 minutes to complete.

The result clearly showed the Picasso colors and texture bleeding into the city skyline. The reds, yellows, and greens from the painting covered the buildings and background while the skyline shape and the ferris wheel were still clearly recognizable. This confirmed that the algorithm was working — the content structure stayed intact and the style transferred successfully.

Run 1 also took the longest out of all runs at 75.63 minutes. This was because I used `max_size=400` instead of 300 like the other runs. That difference alone added about 33 minutes to the runtime which showed that image size has a big impact on how long each run takes.

Figure 4. Run 1 content + result side by side



5.2 Hyperparameter Comparison Results

The table below summarizes the results across all five runs including the content and style images used, hyperparameter settings, final loss, and training time.

Table 1. RESULTS SUMMARY

=====							
STYLE TRANSFER EXPERIMENTS – RESULTS SUMMARY							
=====							
Run	Content	Style	style_weight	max_size	Steps	Final Loss	Time (min)
Run 1	City Skyline	Picasso	1e6	400	2000	13210935.00	75.63
Run 2	Toronto Flatiron	Starry Night	1e6	300	2000	2796110.50	42.35
Run 3	NYC Aerial	Circuit Board	1e6	300	2000	6580532.00	42.60
Run 4	NYC Aerial	Circuit Board	1e4	300	2000	65863.63	42.40
Run 5	NYC Aerial	Circuit Board	1e6	300	500	34327488.00	10.36
=====							

Different Style Images — Runs 1, 2, 3

Runs 1, 2, and 3 all used the same settings (style_weight=1e6, steps=2000) but with different content and style image combinations. The results were noticeably different from each other depending on which style image was used.

Run 1 with the Picasso painting gave bold warm colors with strong geometric texture on the skyline. Run 2 with Starry Night gave a smoother result where Van Gogh's swirling blue brushstrokes blended into the Toronto building details in a natural way. Run 3 with the circuit board gave the most unique result — the teal and orange circuit traces mapped onto the NYC aerial grid making the city look like it was built out of electronics.

Run 2 ended up with the lowest final loss among the runs that used style_weight=1e6 at 2,796,110. . This was likely because the Starry Night and the Toronto photo shared similar structural complexity which made the style transfer more efficient.

Figure 5. Run 2 content + result side by side

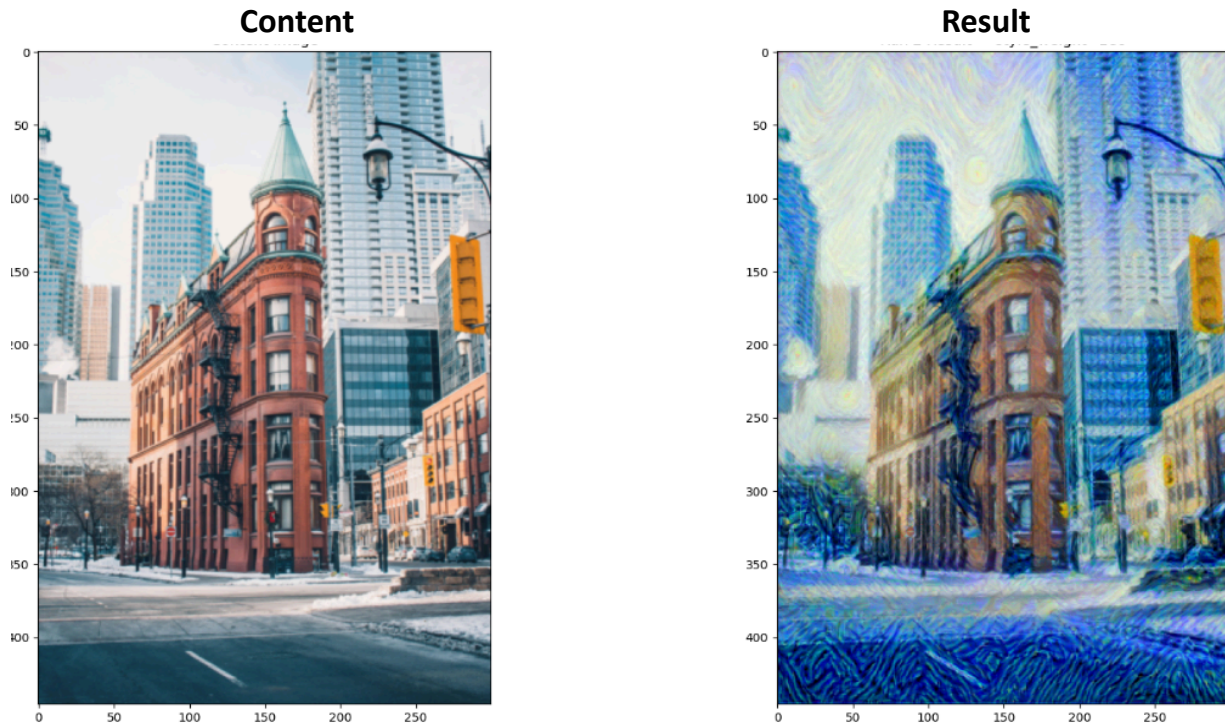


Figure 6. Run 3 content + result side by side

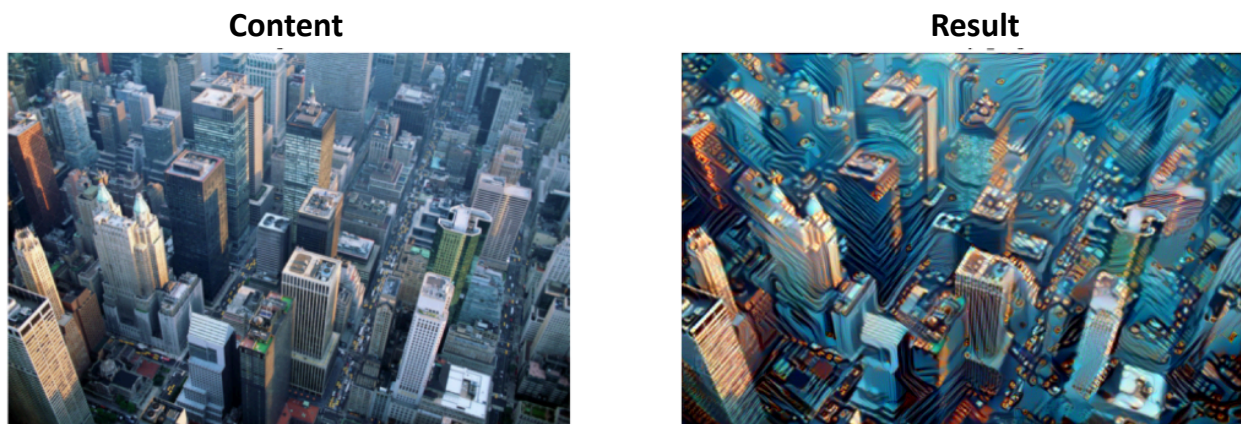


Figure 7. Run 4 content + result side by side

Content



Result



Figure 8. Run 5 content + result side by side

Content



Result



Effect of style_weight — Run 3 vs Run 4

Runs 3 and 4 used the exact same images and settings except for style_weight. Run 3 used $1e6$ and Run 4 used $1e4$. The final losses were 6,580,532 and 65,863 respectively.

Even though Run 4 had a much lower loss, the result actually looked worse visually. The output was almost identical to the original NYC photo with barely any circuit board texture showing up. This showed that lower loss does not always mean a better looking result — it just means the style was barely applied.

Effect of Steps — Run 3 vs Run 5

Runs 3 and 5 used the same images and style_weight= $1e6$ but different step counts. Run 3 used 2000 steps and Run 5 used 500 steps. The final losses were 6,580,532 and 34,327,488 and the runtimes were 42.60 and 10.36 minutes.

Run 5 finished about 4 times faster but the result was noticeably less stylized. This confirmed that more steps lead to better style transfer quality and cutting steps too much is not worth the time savings.

Figure 9. Final Loss and Training Time per Run

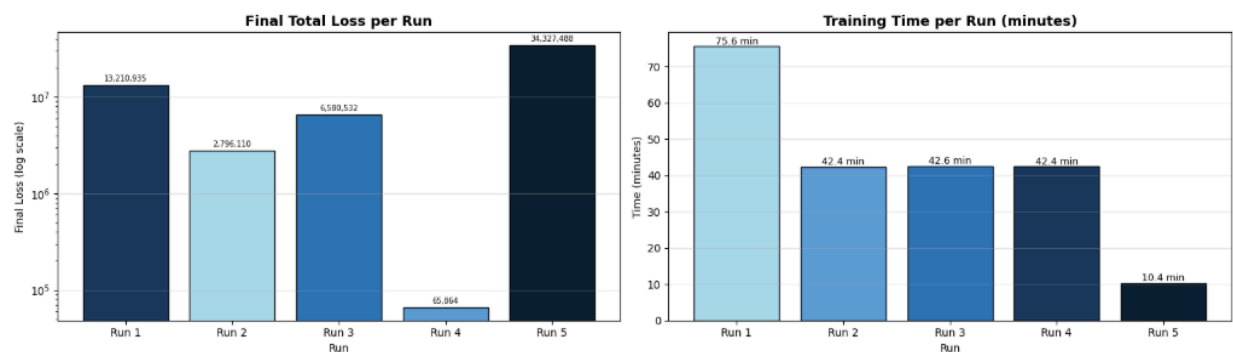
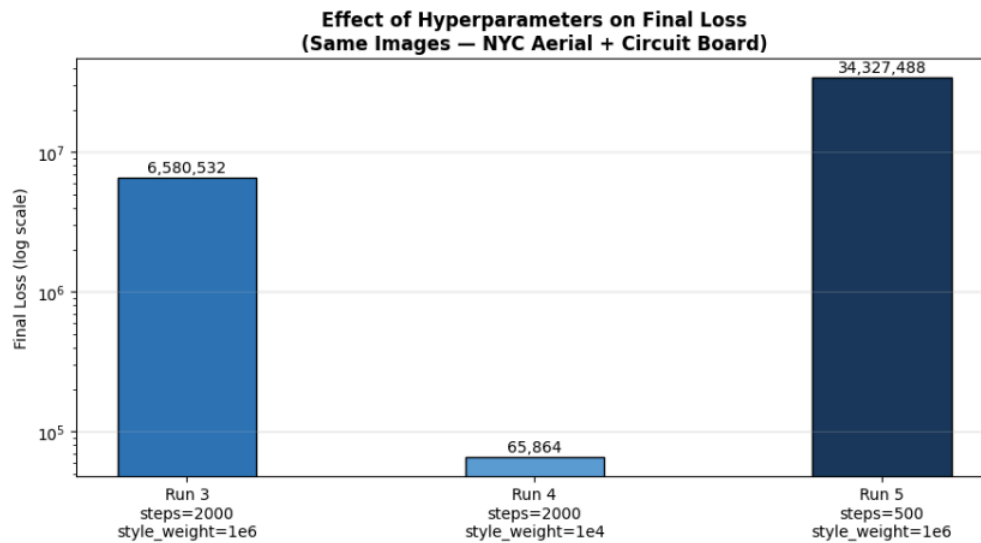


Figure 10. Effect of Hyperparameters on Final Loss — Run 3 vs 4 vs 5



5.3 What Helped and What Did Not

A few things clearly helped get better results. Using a higher `style_weight` made the style transfer much more visible and dramatic. The difference between Run 3 and Run 4 showed this very clearly — dropping `style_weight` from $1e6$ to $1e4$ basically turned off the style transfer. Running more steps also helped a lot. Run 5 at 500 steps was noticeably weaker than Run 3 at 2000 steps with the same settings. Choosing style images with strong distinct textures like the circuit board and Starry Night also worked better because those patterns transferred more cleanly onto the content.

What did not help was lowering `style_weight` too much or cutting steps too aggressively. Both of those changes hurt the visual quality more than they saved time. Large image sizes also made training significantly slower without a proportional improvement in quality — Run 1 at `max_size=400` took 75 minutes while all other runs at `max_size=300` took around 42 minutes.

6. Discussion

6.1 Best Hyperparameter Settings

Based on the five runs, the best overall settings were `style_weight=1e6`, `steps=2000`, and `max_size=300`. This combination produced strong and visually clear style transfer results in a reasonable amount of time. Run 2 and Run 3 both used these settings and gave the best looking outputs out of all the experiments.

`style_weight=1e6` was clearly the right choice for getting visible style transfer. Run 4 showed that dropping to `1e4` basically removed the style effect entirely, so anything lower than `1e6` is not worth using if the goal is actual artistic style transfer.

For steps, 2000 was the minimum needed for a good result. Run 5 showed that 500 steps was not enough — the style was only partially applied and the result looked unfinished. If more time was available, going above 2000 steps would likely improve the result further.

For image size, `max_size=300` was a good balance between quality and speed. Run 1 at `max_size=400` did not look dramatically better than the 300 runs but took almost double the time. For experimentation purposes `max_size=300` is more practical.

6.2 Trade-offs Between Style Quality, Content Preservation, and Runtime

The main trade-off in this algorithm is between style quality and content preservation. These two goals work against each other because they are controlled by the same loss function. A higher `style_weight` pushes the optimizer to match the style gram matrices more aggressively, which means the content structure gets modified more. A lower `style_weight` keeps the content intact but barely applies any style.

In Run 4 this trade-off was very clear. With `style_weight=1e4` the content was almost perfectly preserved but the style was invisible. With `style_weight=1e6` in Run 3 the style was strong but the original NYC photo was noticeably altered. Finding the right balance depends on the specific images being used and what the goal is visually.

Runtime is the other major trade-off. More steps and larger image sizes both improve quality but increase training time significantly. Run 1 at `max_size=400` with 2000 steps took 75 minutes while Run 5 at `max_size=300` with 500 steps took only 10 minutes. The quality difference was visible but for quick experiments or testing different settings the faster configuration is more practical.

Overall the algorithm works best when the style image has strong and distinctive texture patterns. Images like *Starry Night* and the circuit board transferred cleanly because their

textures were well defined. A style image with subtle or low contrast patterns would likely produce weaker results regardless of the hyperparameter settings.

7. Conclusion

7.1 Summary of Findings

Overall this assignment was about implementing neural style transfer based on the Gatys et al. 2016 paper using a pretrained VGG19 network in PyTorch. The main goal was to take a content image and a style image and generate a new image that preserved the structure of the content while applying the visual appearance of the style.

Five runs were completed across three different content and style image combinations. The results showed that the algorithm worked well across all combinations and produced visually interesting outputs. The best results came from using `style_weight=1e6`, `steps=2000`, and `max_size=300`. The most important finding was that `style_weight` had the biggest impact on the visual outcome — lowering it too much essentially disabled the style transfer even though it reduced the loss value significantly. The number of steps also mattered — fewer steps meant less stylization regardless of the other settings.

7.2 Personal Insights

This assignment was different from anything I had done before in this course. In previous assignments the focus was on training a model to classify images. Here the model was never trained at all — the weights were frozen and the optimization happened directly on the pixel values of the image itself. That was a completely different way of thinking about gradient descent and it took a bit of time to fully understand.

I also found it interesting how the choice of style image affected the result more than I expected. The circuit board style produced a completely different kind of output compared to *Starry Night* or the Picasso painting even with the exact same settings. This showed me that the algorithm is very sensitive to the texture patterns in the style image and that picking the right style image matters as much as tuning the hyperparameters.

Another thing I noticed was that a lower loss does not always mean a better result visually. Run 4 had the lowest loss of all runs but the output barely showed any style transfer. That was a good reminder that loss values are just numbers and the actual visual quality of the output is what matters in this kind of task.

7.3 Future Improvements

If I continued working on this there are a few things I would try. First I would experiment with the learning rate to see if a higher or lower value affects convergence speed or final quality. Second I would try changing the per layer style weights to give more emphasis to specific layers and see how that changes the texture scale in the output. Third I would test initializing the target image from random noise instead of the content image to see if that produces more creative results as mentioned in the paper.

I would also try using a GPU instead of CPU which would reduce training time from 40+ minutes per run down to a few minutes. That would make it much more practical to run more experiments and explore a wider range of hyperparameter combinations.

8. References

- Course-provided Jupyter notebook: `Style_Transfer_Exercise.ipynb`, CSCI 611 Applied Machine Learning, Spring 2026.
- CSCI 611 Course Lecture Notes: `CSCI611 Su25 Week4.3 - CNN2StyleTransfer.pdf` (Canvas PDF).
- Gatys, L. A., Ecker, A. S., & Bethge, M. (2016). Image Style Transfer Using Convolutional Neural Networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2414–2423.
- Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv:1409.1556*.
- Machine Learning with Pytorch and Scikit-Learn: Develop Machine Learning and Deep Learning Models with Python.
- CSCI 611 Course Lecture Notes: `CNN.pdf` (Canvas PDF).